# 59. Классы enum

Хотя **перечисления** и считаются отдельными типами данных в языке C++, они не столь безопасны, как кажутся на первый взгляд, и в некоторых случаях позволят вам делать вещи, которые не имеют смысла. Например:

```
#include <iostream>

int main()
{
    enum Fruits
    {
        LEMON, // LEMON находится внутри той же области видимости, что и Fruits
        KIWI
    };

    enum Colors
    {
        PINK, // PINK находится внутри той же области видимости, что и Colors
        GRAY
    };

    Fruits fruit = LEMON; // Fruits и LEMON доступны в одной области видимости (добавлять префикс не нужно)
    Colors color = PINK; // Colors и PINK доступны в одной области видимости (добавлять префикс не нужно)

    if (fruit == color) // компилятор будет сравнивать эти переменные как целые числа
        std::cout << "fruit and color are equal\n"; // и обнаружит, что они равны!
    else
        std::cout << "fruit and color are not equal\n";

    return 0;
}
```

Когда компилятор будет сравнивать переменные `fruit` и `color`, он **неявно преобразует** их в целочисленные значения и сравнит эти целые числа. Так как значениями этих двух переменных являются перечислители, которым присвоено значение `0`, то это означает, что в примере, приведенном выше, `fruit = color`. А это не совсем то, что должно быть, так как `fruit` и `color` из разных перечислений и их вообще нельзя сравнивать (фрукт и цвет!). С обычными перечислителями нет способа предотвратить подобные сравнения.

Для решения этой проблемы в C++11 добавили **классы enum** (или ***«перечисления с областью видимости»***), которые добавляют перечислениям, как вы уже могли понять, **локальную область видимости** со всеми её правилами. Для создания такого класса нужно просто добавить **ключевое слово** `class` сразу после enum. Например:

```
#include <iostream>

int main()
{
    enum class Fruits // добавление "class" к enum определяет перечисление с ограниченной областью видимости, вместо стандарт
    {
        LEMON, // LEMON находится внутри той же области видимости, что и Fruits
        KIWI
    };

    enum class Colors
    {
        PINK, // PINK находится внутри той же области видимости, что и Colors
        GRAY
    };

    Fruits fruit = Fruits::LEMON; // примечание: LEMON напрямую не доступен, мы должны использовать Fruits::LEMON
    Colors color = Colors::PINK; // примечание: PINK напрямую не доступен, мы должны использовать Colors::PINK

    if (fruit == color) // ошибка компиляции, поскольку компилятор не знает, как сравнивать разные типы: Fruits и Colors
        std::cout << "fruit and color are equal\n";
    else
        std::cout << "fruit and color are not equal\n";

    return 0;
}
```

Стандартные перечислители находятся в той же области видимости, что и само перечисление (в **глобальной области видимости**), поэтому вы можете напрямую получить к ним доступ (например, `PINK`). Однако с добавлением класса, который ограничивает область видимости каждого перечислителя областью видимости его перечисления, для доступа к нему потребуется оператор разрешения области видимости (например, `Colors::PINK`). Это значительно снижает риск возникновения **конфликтов имен**.

Поскольку перечислители являются частью класса enum, то необходимость добавлять префиксы к идентификаторам отпадает (например, можно использовать просто `PINK` вместо `COLOR_PINK`, так как `Colors::COLOR_PINK` уже будет лишним).

А строгие правила типов классов enum означают, что каждый класс enum считается уникальным типом. Это означает, что компилятор не сможет сравнивать перечислители из разных перечислений. Если вы попытаетесь это сделать, компилятор выдаст ошибку (как в примере, приведенном выше).

Однако учтите, что вы можете сравнивать перечислители внутри одного класса enum (так как эти перечислители принадлежат одному типу):

```
#include <iostream>

int main()
{
    enum class Colors
    {
        PINK,
        GRAY
    };

    Colors color = Colors::PINK;

    if (color == Colors::PINK) // это нормально
        std::cout << "The color is pink!\n";
    else if (color == Colors::GRAY)
        std::cout << "The color is gray!\n";

    return 0;
}
```

С классами enum компилятор больше не сможет неявно конвертировать значения перечислителей в целые числа. Это хорошо! Но иногда могут быть ситуации, когда нужно будет вернуть эту особенность. В таких случаях вы можете **явно преобразовать** перечислитель класса enum в **тип int**, используя оператор static_cast:

```
#include <iostream>

int main()
{
    enum class Colors
    {
        PINK,
        GRAY
    };

    Colors color = Colors::GRAY;

    std::cout << color; // не будет работать, поскольку нет неявного преобразования в тип int
    std::cout << static_cast<int>(color); // результатом будет 1

    return 0;
}
```

Если вы используете компилятор, поддерживающий C++11, то нет никакого смысла использовать обычные перечисления вместо классов enum.

Обратите внимание, ключевое слово class вместе с ключевым словом static являются одними из самых запутанных в языке C++, поскольку результат сильно зависит от варианта применения (контекста). Хотя классы enum используют ключевое слово class, в C++ они не считаются традиционными «классами». О традиционных классах мы поговорим несколько позже.